

DeploymentWork — your task list

Replit → DigitalOcean, using **Coolify Cloud** (hosted control plane, your own droplet as the deploy target)
+ DO Managed Postgres + DO Spaces.

Two stages: **A** — prove it on a test **subdomain** (e.g. `test.<your-domain>`). **B** — cut over on your real domain. Moving between them is done entirely in the Coolify panel + DNS — nothing in code or the database references the domain. The only setting that carries it is the `FRONTEND_URL` env var (Socket.io CORS), and even that falls back to `*` if forgotten.

Details for any database step live in **MIGRATION.md**.

Contents

1 What your data actually contains

2 Stage A — test deployment (A1–A10)

3 Stage B — cutover on your real domain

4 Already done in the code

5 Environment variables — reference

6 After the soak week

7 Coolify: the exact variables to paste, and where each value comes from

1 What your data actually contains

Item	Value
Size / rows / tables	2.7 GB / 3,160,934 / 120
Foreign keys	252 — all verified, 0 orphans
Files to move to Spaces	2 PDFs . No call recordings.
Inbound lead webhooks	11 active (146k requests)
Google Sheets backups	12 of 12 companies enabled
WhatsApp	via Wauper , 29,854 messages, 2 companies
Push subscriptions / API keys	0 / 0

Three things this changes:

- **File migration is trivial** — two PDFs, not the multi-week job a generic plan assumes.
- **Google Sheets is your riskiest integration**, not WhatsApp. All 12 back up hourly.
- **There is no Meta app to reconfigure.** `whatsapp_cloud_config` and `meta_platform_settings` are empty; WhatsApp runs through Wauper, credentials in the database.

2 Stage A — test deployment

A1. Google service account

Only genuinely new credential. Gates all 12 companies' backups, so start early.

1. Google Cloud console → enable **Google Sheets API**.
2. Service Accounts → create → **Add key (JSON)** → download.
3. **Base64-encode the file** (the private key's newlines break most env-var UIs).
4. **Share all 12 backup spreadsheets with the service account address as Editor**.

Don't set this on Stage A — it goes in at B5.

A2. Check what you already have

Coolify Cloud, the droplet (4 GB / 2 vCPU, two other apps), the Postgres cluster and the Space all exist. Four things to confirm before deploying — run these on the droplet and against the cluster.

Swap — `free -h`, look at the Swap row. If it shows `0B`:

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

On a 4 GB box already running two apps this is the one that matters. Without swap, an OOM during a build can make the kernel kill a **running** app instead of the build.

Postgres version — `psql "$DO_DATABASE_URL" -c 'SHOW server_version;'`

Either way is fine. On 16 you'll see harmless `transaction_timeout` errors during the restore (the script filters them); on 17+ the log is clean.

Connection budget — the cluster is shared with your other two apps, and basic plans allow only ~22 backends *in total*:

```
SELECT current_setting('max_connections') AS limit, count(*) AS in_use FROM pg_stat_activity;
```

Subtract what the other two apps hold. This app opens up to `DB_POOL_MAX` (default 10) plus a few for the restore. If the headroom is tight, set `DB_POOL_MAX=5` — measured idle usage is well under that, and running out of cluster connections takes down all three apps, not just this one.

Space — confirm it is **private** (no public listing), and that you have the endpoint, region, key and secret. The app throws at boot if any `S3_*` is missing.

Database — use a dedicated database on the cluster (not `defaultdb`), and the **direct connection string, not the pooled one**. Saila takes session-level advisory locks; a transaction-mode pooler splits lock and unlock across backends, and the code fails silently rather than erroring — it just leaks locks until connections run out. URL ends `?sslmode=require`. No extensions needed.

A3. Capacity — the build is the only tight spot

Runtime is not a concern: **measured at 138 MB** for the app container against the full production dataset. Three apps on 4 GB is comfortable.

Builds are different — Coolify builds on your droplet, alongside the two running apps. The Dockerfile used to request a 3 GB Node heap; it now defaults to **2048 MB**, measured with headroom (the build completes at 1536 and produces byte-identical output). If a build still OOMs, either raise swap or pass

`NODE_BUILD_HEAP_MB` as a Coolify build argument to tune it.

Also add the **droplet's** IP to the database's **Trusted Sources** if it isn't already there — the app connects from the droplet, not from Coolify's control plane.

A4. Move the two PDFs to Spaces

Run **from the Replit shell** (production `./uploads` is ephemeral Cloud Run storage):

```
S3_ENDPOINT=... S3_REGION=... S3_BUCKET=... S3_ACCESS_KEY_ID=... S3_SECRET_ACCESS_KEY=... \  
DATABASE_URL=<replit-db-url> npx tsx scripts/migrate-uploads-to-s3.ts
```

Must end with **"All notice files present in bucket."**

A5. Restore the database

Run from the droplet. **Schema, then data, and do not start the app in between** — the boot seeder writes 35 rows into an empty `system_value_definitions` and the data restore then duplicates them.

```
# Client tools must be PostgreSQL 18+. v16 cannot read these archives at all.  
TARGET_URL="postgresql://...?sslmode=require" scripts/db-restore.sh leadani-db-schema.sql  
  
EXCLUDE_TABLES="sheet_snapshots snapshot_restore_logs"  
TARGET_URL="postgresql://...?sslmode=require" scripts/db-restore.sh leadani-db-data.sql
```

`sheet_snapshots` is skipped by your decision (27 GB uncompressed; the scheduler rebuilds it).

`snapshot_restore_logs` must go with it — and is empty anyway.

Ignore the `transaction_timeout` **error and the exit code.** Trust the script's `!!! REAL ERRORS` line.

A6. Build indexes — before first boot

```
psql "$DO_DATABASE_URL" -f db/performance-indexes.sql
```

The app creates these at boot too, but on a restored database that's minutes of locked work against a 120s healthcheck. Doing it first makes boot a no-op.

A7. Coolify application settings

Setting	Value
Build pack	Dockerfile
Domain	<code>https://test.<your-domain></code> — see below
Port	5000
Health check path / start period	<code>/health</code> / 120s
Replicas	1 — never change

Deploy strategy	stop old, then start new (not rolling)
Stop grace period	30s
Persistent storage	none

Seven in-process schedulers plus in-memory Socket.io and rate-limit state. A rolling deploy briefly runs two of everything — duplicate backups, double purges. Grace period must exceed the app's 20s drain.

Domain: create a DNS A record `test.<your-domain>` → droplet IP, then put `https://test.<your-domain>` in the app's **Domains** field. Coolify's proxy requests the Let's Encrypt certificate automatically once DNS resolves — nothing to install. Two things the subdomain can't rehearse: logins don't carry across origins (tokens live in per-origin localStorage, so you log in fresh on the subdomain — and real users' sessions on the real domain survive cutover for the same reason), and the 11 inbound webhooks keep pointing at the real domain, so webhook flows are only testable at Stage B step 8.

A8. Environment variables

● The one that matters on a test deployment:

`OUTBOUND_INTEGRATIONS=disabled`

Your test app runs on a copy of production data carrying live Wauper tokens and 12 enabled backup targets. Without this it sends **real WhatsApp messages to real leads** and **overwrites your real spreadsheets**. Not hypothetical: the local stack attempted 312 backup writes against your real spreadsheets in 13 hours — every one failed only because the Google credential happened to be missing.

```
NODE_ENV=production
PORT=5000
DATABASE_URL=          # direct URL, ?sslmode=require
FRONTEND_URL=https://test.<your-domain>
JWT_SECRET=            # byte-identical to Replit
HMAC_SECRET=           # byte-identical to Replit
VAPID_PUBLIC_KEY=
VAPID_PRIVATE_KEY=
SARVAM_API_KEY=
STORAGE_DRIVER=s3
S3_ENDPOINT= S3_REGION= S3_BUCKET= S3_ACCESS_KEY_ID= S3_SECRET_ACCESS_KEY=

DB_POOL_MAX=5          # shared cluster - see A2
OUTBOUND_INTEGRATIONS=disabled # ← Stage A only
```

Omit `GOOGLE_SERVICE_ACCOUNT_JSON` (belt and braces) and `SUPER_ADMIN_EMAIL` / `PASSWORD` (the account is in the restore). Full paste list with sources: **section 7**.

A9. Start and check the log

- ✓ `[Seed] Completed. Created: 0, Skipped: 35` — **proof the restore order held**
- ✓ `[config] OUTBOUND_INTEGRATIONS=disabled` — must be present
- ✓ `[LogRetention] Scheduler initialized`

✓ no SECURITY WARNING: JWT_SECRET is not set

A10. Test, then soak

Log in as a real user; open a big sheet; view **and upload** a notice PDF; open two tabs and confirm live updates propagate.

Check `trust proxy`: log `req.ip` from a known external IP. If it shows Traefik's internal `172.x`, every user collapses onto one IP and the login limiter locks everyone out. Two hops (e.g. Cloudflare) means changing `1` to `2` in `server/app.ts`.

Leave it running a few days. Real users stay on Replit throughout.

3 Stage B — cutover on your real domain

Because the domain doesn't change, **all 11 inbound webhooks and the WhatsApp callback keep working untouched**. Don't give that up by changing the domain late.

1. **Day before:** lower DNS TTL to 300s. Pre-write the rollback DNS record.
2. **Stop writes:** idle the Replit deployment.
3. **Fresh dump + restore** into a **new** database, exact procedure from A5. Don't reuse the Stage A database — it has test writes mixed into real data.
4. **Indexes + stats:** `psql "$DO_DATABASE_URL" -f db/performance-indexes.sql` (ends with `ANALYZE`).
5. **Switch env:** new `DATABASE_URL`, real `FRONTEND_URL`, add `GOOGLE_SERVICE_ACCOUNT_JSON` — and **delete** `OUTBOUND_INTEGRATIONS`.

● Forgetting this is the failure mode of the whole plan: the app looks perfect and silently sends nothing. Confirm by its **absence** from the log.

6. **Add the real domain in Coolify:** in the app's **Domains** field, add `https://<your-domain>` **alongside** the test subdomain (the field takes a comma-separated list) — don't replace it, so the subdomain keeps serving while DNS moves. The certificate for the real domain is issued automatically on the first requests after DNS points at the droplet: expect seconds to a minute of cert warm-up, not downtime.
7. **Start**, check A9's list — this time `OUTBOUND_INTEGRATIONS=disabled` must **not** appear.
8. **Test what Stage A couldn't:** send a WhatsApp message to a real handset; receive an inbound one and confirm Saila replies; run a manual Sheets backup; post to one inbound webhook.
9. **Switch DNS.**

Rollback line: free and lossless until the first user write on DO (revert DNS, restart Replit — same `JWT_SECRET` means nobody is logged out). After that, forward-fix only. Keep Replit stopped but intact for a week.

4 Already done in the code

- **Outbound kill switch** — gates WhatsApp, webhooks, Sheets, push. Logs `[outbound:blocked]`. Defaults to enabled, so production needs no variable.
- **Indexes** — five of the largest tables had *only* a primary key. `outgoing_webhook_logs` (1 GB) and `sheet_snapshots` were sequential-scanned on every lookup, the latter 50× an hour.
- **Log retention** — five unbounded log tables were 2.1 GB of 2.7 GB. Now pruned daily at 90 days (`LOG_RETENTION_DAYS`, `0` disables).

⚠ This **permanently deletes rows**. If `audit_logs` is a compliance record for you, set `LOG_RETENTION_DAYS=0` before B5.

- Earlier: graceful shutdown, `trust_proxy`, S3 driver, host-agnostic Google auth, Replit lockfile URLs fixed.

5 Environment variables — reference

Variable	Notes
<code>DATABASE_URL</code>	Required. Direct URL, <code>?sslmode=require</code> .
<code>JWT_SECRET</code> / <code>HMAC_SECRET</code>	Required. Copy byte-for-byte — changing them logs everyone out / breaks webhook signatures.
<code>FRONTEND_URL</code>	Public https URL. Socket.io CORS origin.
<code>STORAGE_DRIVER</code> + 5× <code>S3_*</code>	<code>s3</code> in production. App refuses to boot if any S3 var is missing.
<code>GOOGLE_SERVICE_ACCOUNT_JSON</code>	Base64. Without it all 12 backups fail hourly. Stage B only.
<code>SARVAM_API_KEY</code>	Optional; also settable per-company in the UI.
<code>VAPID_PUBLIC_KEY</code> / <code>VAPID_PRIVATE_KEY</code>	Optional — currently 0 subscriptions , so low stakes.
<code>OUTBOUND_INTEGRATIONS</code>	<code>disabled</code> on Stage A. Absent in production.
<code>LOG_RETENTION_DAYS</code>	Default 90. <code>0</code> keeps everything.
<code>DB_POOL_MAX</code>	Default 10. Your cluster is shared with two other apps against a ~22 backend limit — use 5.
<code>SUPER_ADMIN_EMAIL</code> / <code>_PASSWORD</code>	Omit — the account is already in the restore.

The secrets you asked about

- `SARVAM_API_KEY`, `VAPID_PUBLIC_KEY`, `VAPID_PRIVATE_KEY` — real, carry them across.
- `SESSION_SECRET` — **not read anywhere in the code**. The app uses JWTs, not `express-session`. Leftover in your Replit secrets; nothing will miss it.
- `WAUPER_API_KEY` — **not read from the environment**. Wauper credentials live in the database (`saila_config.wauper_api_key`, `saila_phone_settings.access_token`) and travel with the dump.

Setting it on DO does nothing — which is why it's worth rotating after cutover.

6 After the soak week

- **Secure the inbound WhatsApp webhook.** `POST /api/public/whatsapp/:companyId` takes any request with no signature check, and its verify endpoint returns `hub.challenge` to anyone. Anyone with a company ID can inject fake messages and trigger real Saila replies at your cost. Same exposure exists on Replit today — the migration doesn't worsen it, but it should be fixed.
- **Rotate** the Wauper tokens (plaintext DB columns that just moved hosts), then `JWT_SECRET` / `HMAC_SECRET` in an announced window. Never during cutover.
- **Make config guards hard errors** (`server/config.ts` only warns today) — after cutover, not before, or a missing variable becomes a boot loop on the worst day.
- **Monitoring:** uptime check on `/health` from outside, plus a log drain. A crash loop on a droplet is invisible until a user complains.
- **Retire the test subdomain:** remove `test.<your-domain>` from Coolify's Domains field and delete its DNS record. Until then it serves the production app on a second origin — harmless, but one more thing answering requests.
- **Clean up Replit residue:** `.replit`, `replit.nix`, `replit.md`, `scripts/post-merge.sh`, the `@replit/vite-plugin-*` devDependencies.

Schema changes from now on

Replit ran `npm run db:push` automatically on merge. **Coolify has no equivalent and you shouldn't build one.** Do it manually: add your IP to Trusted Sources → `DATABASE_URL=<do-url> npm run db:push` → **read the diff** → remove your IP.

⚠ Read that diff carefully. The database has **141 indexes**; `shared/schema.ts` **declares 7**. Drizzle may offer to drop the rest, including the performance indexes above. This hazard predates the migration.

7 Coolify: the exact variables to paste, and where each value comes from

Enter these in Coolify → your app → **Environment Variables**. Mark the secrets (🔒 below) as *Locked* so they don't show in build logs.

Stage A — set all of these

Variable	Value / where you get it
<code>NODE_ENV</code>	Literal: <code>production</code>
<code>PORT</code>	Literal: <code>5000</code>
<code>DATABASE_URL</code> 🔒	DO panel → Databases → your cluster → Connection details → pick your dedicated database (not <code>defaultdb</code>) → Connection string , and choose the direct/public connection, NOT the connection pooler (Saila's advisory locks break through a pooler — see A2). Must end <code>?sslmode=require</code> .

DB_POOL_MAX	Literal: <code>5</code> — the cluster is shared with two other apps against a ~22 backend limit (A2).
FRONTEND_URL	You write it: <code>https://test.<your-domain></code> on Stage A → change to <code>https://<your-domain></code> at B5.
JWT_SECRET 	Replit → your Repl → Tools → Secrets . If it's not there (expected — the current deployment runs on the built-in fallback, per the note in <code>server/config.ts</code>), set it to the fallback verbatim: <code>dabluz-crm-secret-key-change-in-production</code> . That keeps every existing login working. Rotate after the soak week, never at cutover.
HMAC_SECRET 	Same as above. Fallback if absent in Replit Secrets: <code>dabluz-webhook-secret-change-in-production</code> . Keeps webhook signatures valid.
STORAGE_DRIVER	Literal: <code>s3</code> (the app refuses to boot if any S3 var below is then missing — that's intentional).
S3_ENDPOINT	DO panel → Spaces → your Space → the Origin Endpoint shown on the bucket page, e.g. <code>https://<region>.digitaloceanspaces.com</code> (endpoint without the bucket name in it).
S3_REGION	The region slug in that endpoint (e.g. <code>blr1</code> , <code>sgp1</code>).
S3_BUCKET	The Space's name (first path/subdomain segment on the bucket page).
S3_ACCESS_KEY_ID 	DO panel → API (left nav) → Spaces Keys tab → <i>Generate New Key</i> .
S3_SECRET_ACCESS_KEY 	Shown once when that key is generated — save it immediately. If lost, generate a new key pair.
SARVAM_API_KEY 	Replit → Tools → Secrets — copy across. (Optional; also settable per-company in the UI.)
VAPID_PUBLIC_KEY / VAPID_PRIVATE_KEY 	Replit → Tools → Secrets — copy across. Low stakes: 0 push subscriptions today.
OUTBOUND_INTEGRATIONS	Literal: <code>disabled</code> — Stage A only (see the red box at A8).

Stage B — the only changes (B5)

Variable	Change
DATABASE_URL	Point at the new database from the fresh cutover restore (B3).
FRONTEND_URL	<code>https://<your-domain></code>
GOOGLE_SERVICE_ACCOUNT_JSON 	Add now, not on Stage A. The base64 of the service-account key file you downloaded in A1 (Google Cloud console → Service Accounts → Add key → JSON). Encode with <code>base64 -w0 key.json</code> (or PowerShell: <code>[Convert]::ToBase64String([IO.File]::ReadAllBytes("key.json"))</code>).

`OUTBOUND_INTEGRATIONS`

Delete the variable entirely. Confirm by its absence from the boot log.

Optional

Variable	Notes
<code>LOG_RETENTION_DAYS</code>	Default <code>90</code> with no variable set. Set <code>0</code> before B5 if <code>audit_logs</code> is a compliance record you must keep forever.
<code>DATABASE_CA_CERT</code> 	Hardening, not required: DO panel → your database → Connection details → Download CA certificate . Paste the PEM or its base64 — the app accepts both (base64 avoids multiline env-var trouble). Makes the app verify the DB's TLS cert instead of just encrypting. Fine to skip for cutover.

Do NOT set

- `SUPER_ADMIN_EMAIL` / `SUPER_ADMIN_PASSWORD` — the account is already in the restore; setting them just re-runs seeding logic you don't need.
- `SESSION_SECRET` — not read anywhere in the code (Replit leftover).
- `WAUPER_API_KEY` — not read from the environment; Wauper credentials live in the database and travel with the dump.
- `S3_FORCE_PATH_STYLE` — leave unset; the default (`false`) is correct for DO Spaces.